DEV 140 · DISCUSSION 2 · HOMEWORK 1

Build Your Homework 1 Page

TODAY'S PLAN

Work on HW 1

1

Get the template

Copy two files from the class folder on GitHub.

2

Preview live

Live Server shows every save in your browser.

3

Build it

Fill the template with your own topic and 5+ semantic elements.

4

Push & submit

Commit, sync, and post your repo on Campuswire.

HTML is plain text that labels your content

You write this in VS Code

```
<h1>Sourdough 101</h1>
<p>Bread made from flour,
water and salt.</p>
```

The browser shows this



Sourdough 101

Bread made from flour, water and salt.

The **tags** in angle brackets never appear on the page. They tell the browser what each piece of text *is*: a heading, a paragraph.

Most elements are a pair of *tags* around content




```
<h2>My Method</h2>
```

```
<li>Flour</li>
```

```
<footer>…</footer>
```

Opening tag + content + closing tag = an **element**. The closing tag repeats the name with a / in front.

HTML BASICS · 3 OF 5

Attributes add details inside the opening tag



The pattern: name="value"

Separated by spaces, value in double quotes. An element can have several.

Links use href

```
<a href="#about">About</a>
```

No closing tag

<img> has no content, so it has no </img>. Same for <meta> and
.

Elements go inside other elements

Code: indent what's inside

```
<ul>
  <li>Flour</li>
  <li>Water</li>
  <li>Salt</li>
</ul>
```

Browser shows

- Flour
- Water
- Salt

Wrong: the tags cross

```
<p><em>Hi</p></em>
```

Right: close the inner one first

```
<p><em>Hi</em></p>
```

HTML BASICS · 5 OF 5

Every page has the same skeleton

```
<!DOCTYPE html>                    ← "this is a modern HTML page"
<html lang="en">                   ← wraps everything
  <head>                           ← head: info about the page
    <title>Sourdough 101</title>   ← text on the browser tab
  </head>
  <body>                           ← body: everything you see
    <h1>Sourdough 101</h1>         ← your content goes here
    <!-- a note to yourself -->   ← a comment: the browser ignores it
  </body>
</html>
```

Type **!** then **Tab** in VS Code to get this for free. The template's instructions are **comments**: write your HTML under each one.

THE BIG IDEA

Semantic tags say what content *is*

Generic: says nothing

```
<div>
  <div>Sourdough 101</div>
  <div>About · Method</div>
</div>
```

Semantic: says what each part is

```
<header>
  <h1>Sourdough 101</h1>
  <nav>About · Method</nav>
</header>
```

Both look the same. Only one tells browsers, screen readers and graders what's what. You need at least 5 different ones.

MUST HAVE · EVERY PAGE

The elements every page needs

Element	What it's for	Example
<!DOCTYPE> & co.	Skeleton: html, head, body	Type ! then Tab
<title>	Text on the browser tab	<title>Sourdough Notes</title>
<h1>	Page title: exactly one	Sourdough 101
<h2> <h3>	Sections, then subsections	The Basics
	A paragraph of text	I started baking…
<img>	An image; alt describes it	<img src="…" alt="…">
	A list; ol if order matters	Mix

USEFUL EXTRAS · PICK 3 OR MORE

<p> + your list = 2. Add 3 more to reach 5.

<header>

Top intro: your h1 and nav

<nav>

Links that jump to your sections

<main>

The main content, once per page

<section>

A group with its own h2

<article>

A self-contained piece, like a card

<figure>

An image plus its <figcaption>

<footer>

Closing info: your name, the year

Also handy

`<a>` `<strong>` `<em>`
`<blockquote>` `<aside>`

TWO THINGS GRADERS CHECK FIRST

Headings form an outline. Alt text describes.

```
h1 Sourdough for Beginners
  h2 The Basics
    h3 The Starter
    h3 Hydration
  h2 My Method
```

One h1. Never skip a level. Never pick a heading for its size.



Too vague: partial credit

```
alt="bread"
```

Descriptive: full credit

```
alt="A round, golden-brown sourdough loaf
with a cross cut on top, on a wooden board"
```

STOP 1 · GET THE TEMPLATE

Two files, copied by hand

1. **Open the class folder:** discussion-2 on GitHub
2. **On your PC, make a folder** called discussion-2 inside Discussions, next to discussion-1.
3. **Download the image:** click sourdough-loaf.svg → **Download raw file**, then move it into discussion-2.
4. **Open Discussions in VSCode** (not discussion-2), so Source Control still sees your repo.
5. **Paste the template:** open template.html → **Copy raw file**. In VS Code, new file in discussion-2 named **index.html**, paste, save.

Use GitHub's buttons. Right-click → Save link as saves GitHub's web page, not the image.

STOPS 2 & 3 · PREVIEW LIVE, THEN BUILD

Save, and your browser updates

ONCE

Install Live Server

Extensions (Ctrl+Shift+X) → search Live Server by Ritwick Dey → Install.

EACH SESSION

Click Go Live

Bottom-right status bar. If you see a folder list, click discussion-2/.

AS YOU BUILD

Fill, save, check

Write HTML under each template comment. Ctrl+S refreshes the page.



STOP 4 · PUSH & SUBMIT

Same routine as Discussion 1

Commit

Source Control, message "Add HW1 page", Commit.

Sync

Sync Changes. GitHub now shows discussion-2.

Check

Green ✓ in Actions, then open your live link.

Submit

Post your repo URL and live link on Campuswire.

Your live link

`https://YOUR-USERNAME.github.io/dev-140-yourname-f26/discussion-2/`

HW 1 Rubric · 50 POINTS

Five checks, 10 points each

Criterion	Full credit looks like	Full	Partial	None
Semantic elements	5+ different, each matching its content	10	6	0
Heading hierarchy	One h1, h2 sections, h3 inside, no skips	10	6	0
Image with alt text	Alt describes what the image shows	10	6	0
List element	ul or ol for content that really is a list	10	none	0
Document structure	Doctype, html, head with a real title, body	10	5	0

Biggest losses: a page that won't load, a **private** repo, or a page built from s. The full rubric is on page 2 of your cheat sheet.